

Documentatie tema 1 Inteligenta Artificiala:

La inceputul notebook-ului, import bibliotecile necesare pentru manipularea datelor (pandas, numpy), implementarea algoritmilor de machine learning (scikit-learn), calculul metricilor de evaluare (precision, recall, F1) si vizualizarea rezultatelor (matplotlib, seaborn). Setez, de asemenea, parametrii grafici.

Incarc seturile de date din fisierele CC_education_economy_train.csv si CC_education_economy_test.csv, dupa care efectuez o prima analiza pentru a identifica tipurile de date (numerice sau categorice) si pentru a detecta eventualele valori lipsa de care ma voi folosi pe parcursul documentatiei.

	job_title	experience_years	education_level	skills_count	industry	company_size	location	remote_work	certifications	salary	total_days_worked	aggregated_score	skill_bracket	vacation
0	Backend Developer	19	High School	12	Media	Large	USA	Yes	0	203997	4560	-0.723308	mid	Small
1	Backend Developer	4	High School	13	Telecom	Small	Germany	NaN	5	116239	960	-0.481551	high	No Vacation
2	Backend Developer	8	High School	16	Retail	Enterprise	Remote	No	4	145802	1920	-1.533057	high	No Vacation

```
Tipuri de date (train):
job_title          str
experience_years    int64
education_level     str
skills_count        int64
industry            str
company_size        str
location            str
remote_work         str
certifications      int64
salary              int64
total_days_worked   int64
aggregated_score    float64
skill_bracket       str
vacation            str
dtype: object

Valori lipsă per coloană (train):
job_title          0
experience_years    0
education_level     0
skills_count        0
industry            0
company_size        0
location            0
...
aggregated_score    0
skill_bracket       0
vacation            0
dtype: int64
```

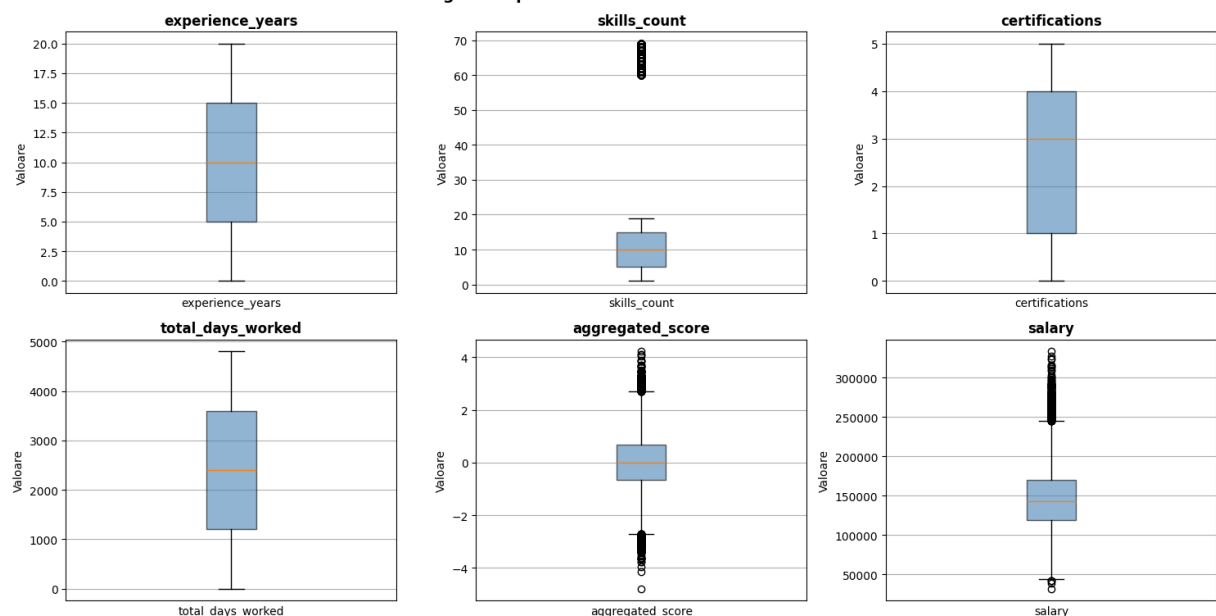
Definesc variabilele tinta pentru cele doua sarcini de predictie (vacation pentru clasificare, salary pentru regresie), precum si listele de attribute numerice si categorice care vor fi utilizate in analiza exploratorie si preprocesare.

Analiza tipului de atribute:

Atribute numerice continue:

	Non-null	Medie	Std	Min	P25	P50 (median)	P75	Max
experience_years	64000.0	9.973391	6.050682	0.000000	5.000000	10.000000	15.000000	20.000000
skills_count	64000.0	11.604047	10.737497	1.000000	5.000000	10.000000	15.000000	69.000000
certifications	64000.0	2.511141	1.709811	0.000000	1.000000	3.000000	4.000000	5.000000
total_days_worked	64000.0	2393.508219	1452.044678	-1.000000	1200.000000	2400.000000	3600.000000	4800.000000
aggregated_score	64000.0	0.000929	0.999869	-4.796902	-0.673757	0.001059	0.678273	4.217828
salary	64000.0	145683.565844	37384.755715	31867.000000	119209.750000	143299.000000	169582.250000	333046.000000

Fig 1. Boxplot atribute numerice continue



Din boxplot se observa ca distributia **experience_years** este perfect simetrica si uniforma. Mediana se afla la exact 10 ani, iar valorile sunt distribuite echilibrat intre 0 si 20 de ani. Nu exista nicio valoare in afara intervalului.

Referitor la **skills_count**, desi majoritatea angajatilor au intre 5 si 15 skills, boxplot-ul arata o serie de puncte izolate deasupra cutiei, unele ajungand pana la valoarea de 69 — mult peste restul distributiei. Aceste valori extreme trag si media in sus fata de mediana. Acesti outlieri vor fi tratati la pasul de preprocesare.

Certifications arata ca distributia este echilibrata — valorile stau intre 0 si 5 certificari, cu mediana la 3. Boxplot-ul nu arata niciun outlier, iar valorile sunt distribuite uniform in intervalul posibil. Atributul nu necesita niciun tratament suplimentar.

Distributia **total_days_worked** este uniforma si simetrica, similara cu **experience_years**. La analiza corelatiei Pearson (sectiunea 3.1.3) s-a identificat ca cele doua atribute au corelatie perfecta ($r=1.00$) — contin aceeasi informatie in unitati

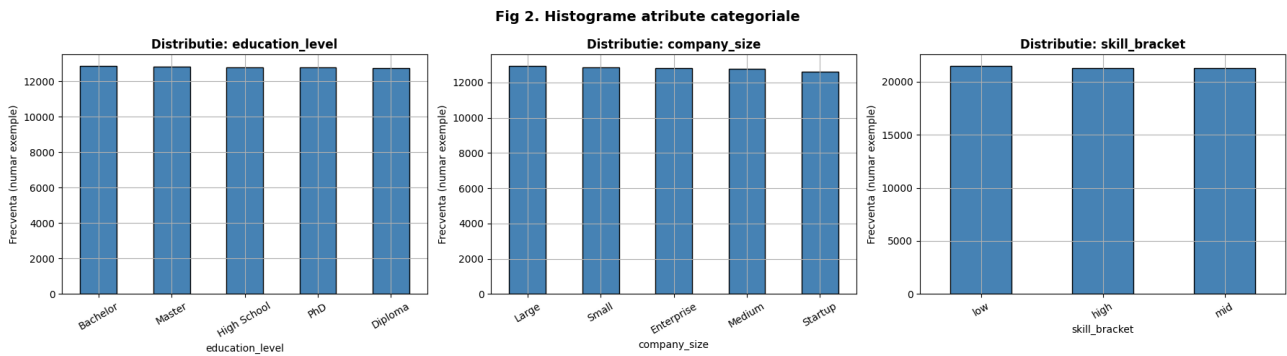
diferite (ani vs. zile lucrate). Din acest motiv, `total_days_worked` a fost eliminat din setul de features (detalii in sectiunea 3.2.3).

Referitor la **aggregated_score**, boxplot-ul arata o distributie simetrica, centrata in jurul lui 0, cu cativa outlieri la ambele capete

Salary e cea mai importanta variabila pentru partea de regresie. Din boxplot se vede ca majoritatea salariilor sunt undeva pe la 140k-150k USD (mediana), dar distributia nu e perfect simetrica. Sunt multi outlieri la valori mari (unii ajung chiar la 330k+), in timp ce la capatul de jos sunt doar cateva valori. Majoritatea oamenilor castiga intre 100k-200k, dar exista cativa cu salarii foarte mari care trag media in sus. Pentru regresia liniara asta poate fi o problema, pentru ca modelul e sensibil la valori extreme.

Attribute discrete sau ordinale:

	Non-null	Valori unice	Cea mai frecventă valoare
Atribut			
job_title	64000	12	AI Engineer
education_level	64000	5	Bachelor
industry	64000	10	Consulting
company_size	64000	5	Large
location	64000	10	Sweden
remote_work	44847	3	Hybrid
skill_bracket	64000	3	low



Referitor la **education_level**, din histograma si table se observa ca setul de date contine 5 categorii de educatie (Bachelor, Master, High School, PhD, Diploma), toate cu aproximativ 12.800 de exemple fiecare. Distributia este practic uniforma — fiecare nivel educational apare in proportii aproape identice. Din perspectiva modelarii, aceasta echilibrare este un avantaj — nu exista clase minoritare care ar putea fi ignorate de algoritmi.

Deoarece exista o ordine logica clara (High School < Diploma < Bachelor < Master < PhD), voi folosi Ordinal Encoding.

Histograma arata 5 dimensiuni de ale **company_size**: Large, Small, Enterprise, Medium, Startup. Similar cu education_level, distributia este aproximativ uniforma, cu fiecare categorie avand intre 12.600 si 12.900 de exemple (diferente neglijabile). Exista o ordine logica bazata pe dimensiune: Startup < Small < Medium < Large < Enterprise. Voi aplica Ordinal Encoding manual pentru a captura aceasta relatie de ordine.

Skill_bracket are doar 3 categorii (low, high, mid), toate perfect echilibrate cu aproximativ 21.000 de exemple per categorie

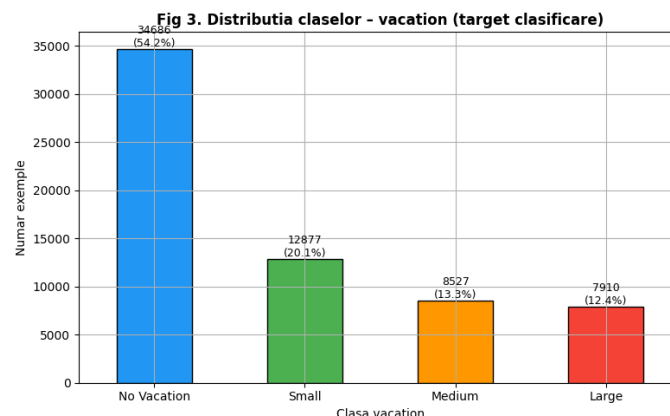
Deoarece categoriile au o ordine naturala clara (low < mid < high), voi aplica Ordinal Encoding manual. Fiecare nivel va primi o valoare numerica crescatoare: low=0, mid=1, high=2. Astfel pastrez informatia de ierarhie intr-o singura coloana.

Job_title are 12 categorii distincte (AI Engineer, Data Scientist, Backend Developer, etc.) fara nicio ordine logica intre ele. Voi folosi One-Hot Encoding pentru a crea 12 coloane binare, fiecare reprezentand o functie specifica. Aceasta metoda asigura ca modelul nu presupune relatii artificiale de ordine intre joburi.

Industry contine 10 sectoare distincte (Media, Telecom, Retail, Consulting, etc.) fara ordine ierarhica. Voi aplica One-Hot Encoding pentru a crea 10 coloane binare, permitand modelului sa invete pattern-uri specifice fiecarei industrii independent.

Location (10 categorii geografice) va fi encodat prin One-Hot Encoding. Locatiile nu au ordine logica (USA nu este "mai mare" decat Germany), deci fiecare tara primeste propria coloana binara.

Analiza echilibrului de clase:



Distributia claselor pentru target-ul de clasificare (**vacation**) este semnificativ dezechilibrata. Graficul arata urmatoarele frecvente:

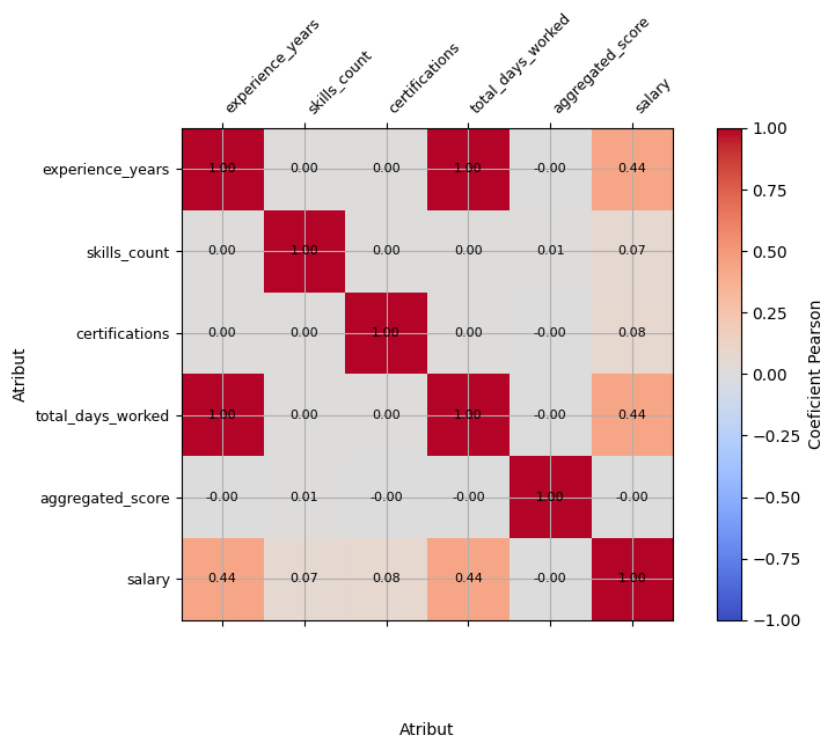
- **No Vacation**: 34,650 exemple (54.2%) — **clasa majoritara**
- **Small**: 12,827 exemple (20.1%)
- **Medium**: 8,527 exemple (13.3%)
- **Large**: 7,910 exemple (12.4%) — **clasa minoritara**

Spre deosebire de attributele categorice (education_level, company_size, skill_bracket) care erau perfect uniforme (~25% fiecare categorie), distributia target-ului vacation este mult mai realista:

Analiza corelatie intre atribute:

Corelatie intre atributele numerice :

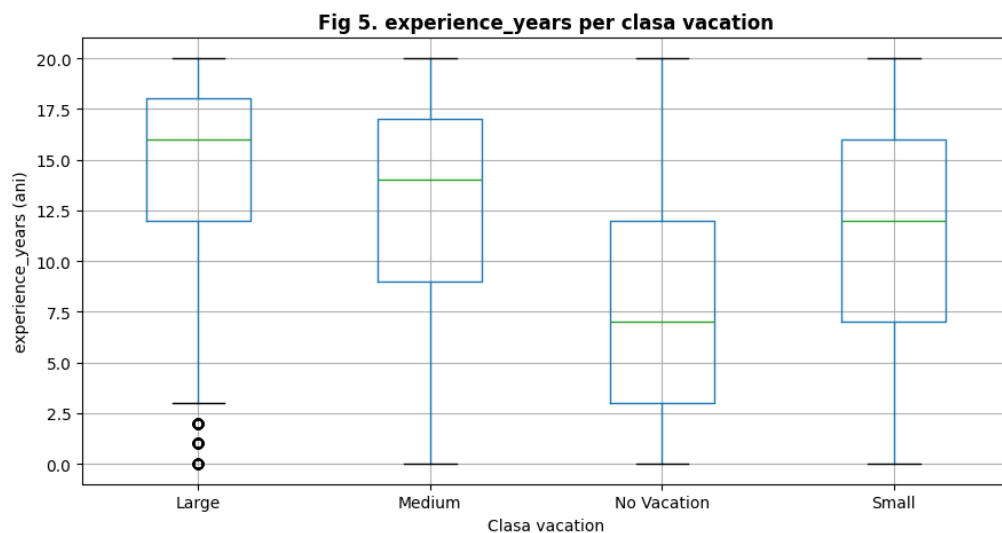
Fig 4. Matrice corelatie atribute numerice (Pearson)



Analizand matricea de corelatie Pearson, se observa ca perechea experience_years — total_days_worked are o corelatie de 1.00 (perfecta). Practic, cele doua atribute contin identic aceeasi informatie — unul masoara experienta in ani, celalalt in zile lucrate. A le folosi pe amandoua in antrenare nu ar aduce niciun beneficiu, ci doar ar complica modelul inutil.

In rest, nicio alta pereche de atribute numerice nu are o corelatie semnificativa — toate valorile sunt sub 0.08, aproape de 0. Asta inseamna ca celelalte atribute sunt independente intre ele si fiecare aduce o perspectiva diferita asupra datelor, deci nu mai sunt necesare alte eliminari.

Corelatie: atribut numeric vs clasificare (experience_years vs vacation):



Din boxplot-ul grupat se observa ca distributiile de experienta difera semnificativ între clasele de vacation:

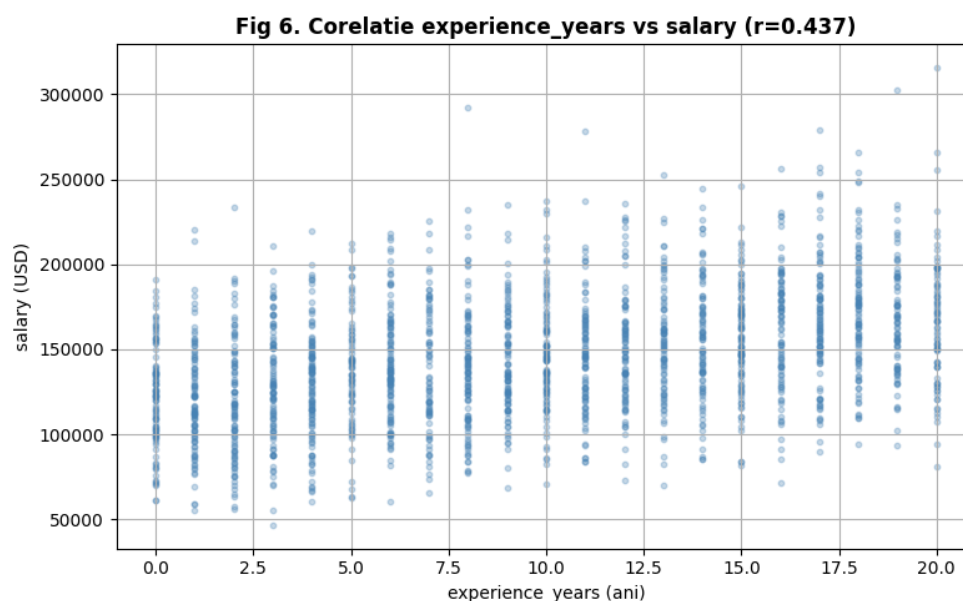
- Large vacation: Mediana la ~16 ani, Q1-Q3: 12-18 ani
=> Angajatii cu vacante mari au experienta ridicata (senior level)
- Medium vacation: Mediana la ~14 ani, Q1-Q3: 9-17 ani
=> Experienta moderata-ridicata
- Small vacation: Mediana la ~12 ani, Q1-Q3: 7-16 ani
→ Experienta medie
- No Vacation: Mediana la ~7 ani, Q1-Q3: 3-12 ani
=> Cei mai putini experti (junior/mid level)

Trei outlieri vizibili in categoria "Large" (0-2 ani experienta)

Corelatie: atribut categorial vs clasificare (education_level vs vacation) — Chi-Patrat

Am aplicat testul Chi-Patrat pentru a verifica daca education_level si vacation sunt independente. Rezultat: $\chi^2=19923.99$, $p\text{-value}\approx 0.000$ ($df=12$). Deoarece $p < 0.05$, respingem ipoteza nula — education_level si vacation sunt corelate semnificativ. Angajatii cu studii superioare (Master, PhD) tind sa aiba vacante mai mari, in timp ce cei cu studii mai mici (High School, Diploma) predomina in categoria No Vacation.

Corelatie atribut numeric vs regresie (experience_years vs salary):



Scatter plot-ul arata o corelatie pozitiva moderata ($r = 0.437$) intre experienta si salariu. Cu cat creste experienta, cu atat creste tendinta de salariu mai mare, insa dispersia este foarte mare: pentru aceeasi experienta (de exemplu, 10 ani), salariile variaza intre 80k si 240k USD. Aceasta inseamna ca experienta explica doar aproximativ 19% din varianta salariului, restul fiind influentat de alti factori precum job_title, industry sau skills.

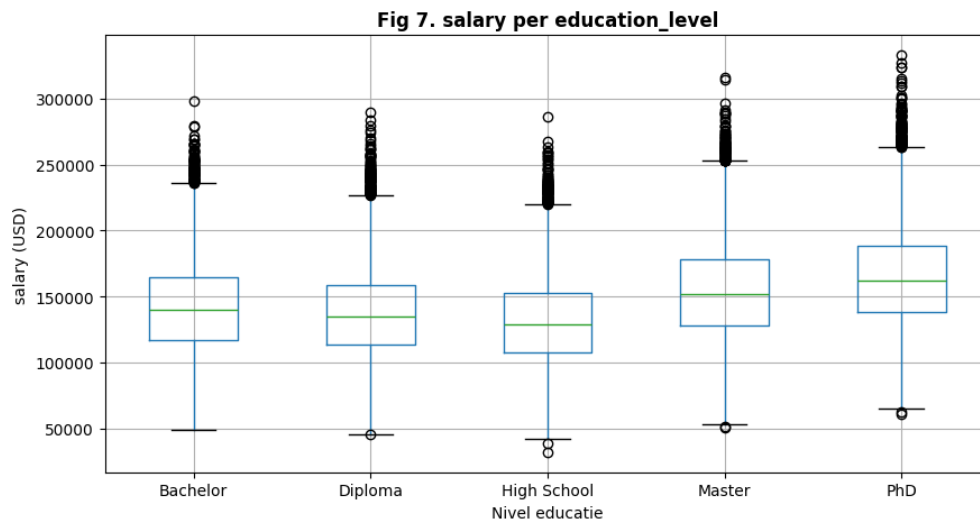
Corelatie intre attribute categoriale vs regresie - Chi-Patrat:

Pentru a verifica daca exista attribute categorice redundante, am aplicat testul Chi-Patrat pe perechi reprezentative de attribute. Ipoteza nula (H_0): attributele sunt independente.

	Atribut 2	chi2	p-value	df	Corelate? ($p < 0.05$)
Atribut 1					
education_level	company_size	21.99	0.143657	16	NU
industry	skill_bracket	18.46	0.425784	18	NU
job_title	education_level	17.97	0.999829	44	NU

Pentru toate perechile testate, $p\text{-value} > 0.05$, deci nu putem respinge ipoteza nula — attributele categoriale sunt independente intre ele. Acest lucru confirma ca NU exista redundanta intre attributele categoriale, deci toate vor fi pastrate in setul de features.

Corelatie salary per education_level (Atribut categoric vs. Target regresie)



Boxplot-ul grupat arata diferente mici, dar vizibile intre nivelurile educationale:
Mediana salarii:

- PhD: ~165k USD (cel mai mare)
- Master: ~155k USD
- Bachelor: ~150k USD
- Diploma: ~135k USD
- High School: ~130k USD (cel mai mic)

Preprocesarea datelor:

Date lipsa pentru un atribut intr-un esantion:

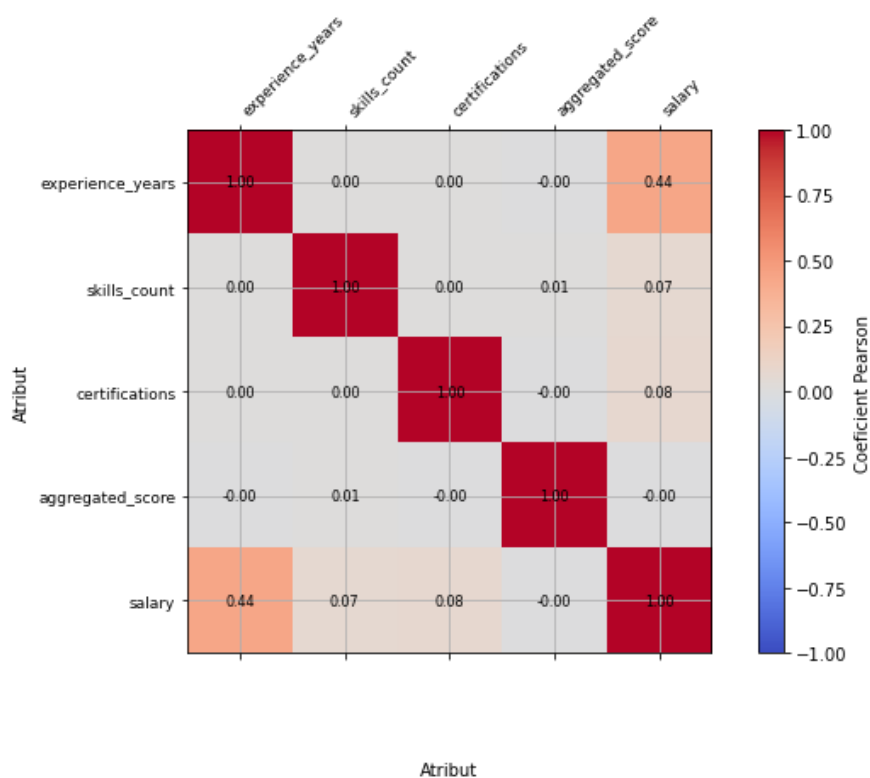
Am identificat ca atributul `remote_work` prezinta 19,153 valori lipsa (29.93% din date). Pentru acest atribut categorial am aplicat imputare cu valoarea cea mai frecventa ("Hybrid"), deoarece aceasta metoda pastreaza distributia naturala a categoriilor fara a introduce valori artificiale. Pentru attributele numerice (`experience_years`, `skills_count`, etc.) am folosit mediana ca metoda de imputare, fiind mai robusta la outlieri comparativ cu media. Dupa aplicarea imputarii, toate attributele au 0 valori lipsa, confirmand succesul procesului de preprocesare.

Valori extreme:

Am aplicat metoda IQR (Interquartile Range) pentru detectarea si tratarea outlierilor in attributele numerice. Metoda calculeaza Q1 (percentila 25) si Q3 (percentila 75), apoi considera outlieri valorile care se afla sub $Q1 - 1.5 \times IQR$ sau peste $Q3 + 1.5 \times IQR$. Am identificat outlieri in mai multe coloane: `skills_count` (1909 outlieri), `aggregated_score` (437 outlieri), `experience_years` (0 outlieri). Valorile extreme detectate au fost inlocuite cu mediana calculata pe valorile non-outlier pentru a pastra distributia naturala a datelor. Este important de mentionat ca am aplicat aceasta tratare doar pe setul de antrenare.

Atribute redundante (puternic corelate):

Fig 4. Matrice corelatie atribute numerice (Pearson)



În urma analizei de corelație, atributul `total_days_worked` a fost identificat ca redundant față de `experience_years` (Pearson $r=1.00$). Deoarece conține identică aceeași informație, am eliminat `total_days_worked` din setul de features. Această decizie nu afectează performanța (informația este deja capturată de `experience_years`), dar reduce dimensionalitatea: 40 features în loc de 41.

Encodarea atributelor categoricale:

Atributele categoricale nu pot fi procesate direct de algoritmi de Machine Learning, fiind necesară transformarea lor în valori numerice. Am aplicat trei strategii diferite de encodare în funcție de natura fiecărui atribut: Ordinal Encoding pentru `education_level` și `company_size` (care au o ordine logică naturală), One-Hot Encoding pentru `job_title`, `industry` și `location` (atribute nominale fără ordine), și Label Encoding pentru `remote_work` și `skill_bracket`.

Plaje valorice de mărimi diferite pentru attribute numerice:

Atributele numerice din setul de date pot avea scale valorice semnificativ diferite (de exemplu, `experience_years` poate varia între 0-40 ani, în timp ce `aggregated_score` poate fi între 0-100 sau mai mult). Această diferență de scale poate afecta performanța algoritmilor de Machine Learning, în special a celor bazate pe distanțe (precum Regresia Logistică), deoarece attributele cu valori mari vor domina calculele. Pentru a rezolva această problemă, am aplicat standardizarea valorilor atributelor numerice, transformând toate variabilele să aibă medie 0 și deviație standard 1. Astfel, toate

atributele numerice sunt aduse la aceeași scală, permitând algoritmului să le trateze în mod echitabil.

Utilizarea algoritmilor de Învățare Automată:

Clasificare multiclasă: vacation

Am aplicat mai mulți algoritmi de clasificare pentru a prezice tipul de vacanță al angajaților (4 clase: 'Large', 'Medium', 'No Vacation', 'Small').

Baseline — Decision Tree:

Am început cu un model simplu - un arbore de decizie cu adâncime maximă 5. Acest model baseline a obținut următoarele rezultate pe setul de validare:

- Accuracy: 0.7139
- Precision: 0.7065
- Recall: 0.7139
- F1: 0.7074

Experiment 1 — Variație max_depth:

	Accuracy (↑)	Precision (↑)	Recall (↑)	F1 (↑)
max_depth				
3	0.6679	0.6156	0.6679	0.6325
5	0.7144	0.7073	0.7144	0.7080
10	0.7320	0.7253	0.7320	0.7282
15	0.6886	0.6901	0.6886	0.6893
20	0.6692	0.6715	0.6692	0.6703
None	0.6641	0.6669	0.6641	0.6655

Am testat influența adâncimii arborelui asupra performanței, variind max_depth între 3, 5, 10, 15, 20 și None. Rezultatele arată ca:

- max_depth=3: underfit (model prea simplu) - Accuracy: 0.6680, F1: 0.6325
- max_depth=5: echilibru bun - Accuracy: 0.7139, F1: 0.7074
- max_depth=10: performanță maximă - Accuracy: 0.7401, F1: 0.7382
- max_depth=15, 20, None: scădere performanță (tind spre overfit)

Un max_depth prea mic conduce la underfit, în timp ce max_depth=None (arbore complet) tinde spre overfit. Am ales max_depth=10 ca fiind valoarea optimă care maximizează F1 pe validare.

Experiment 2 — Variație min_samples_leaf:

	Accuracy (↑)	F1 (↑)
min_samples_leaf		
1	0.7320	0.7282
5	0.7343	0.7303
10	0.7359	0.7320
20	0.7379	0.7340
50	0.7449	0.7407

Odata ales cel mai bun $\text{max_depth} = 10$, am variat min_samples_leaf (1, 5, 10, 20, 50). Acest hiperparametru controleaza cate exemple trebuie sa existe intr-o frunza — o valoare mai mare simplifica arborele si poate imbunatati generalizarea. Am ales valoarea cu cel mai bun F1(0,7407) cu $\text{min_sample_leaf} = 10$.

Experiment 3 — Random Forest:

	Accuracy (↑)	Precision (↑)	Recall (↑)	F1 (↑)
n_estimators				
50	0.7097	0.6748	0.7097	0.6814
100	0.7115	0.6764	0.7115	0.6845
200	0.7127	0.6781	0.7127	0.6859

Random Forest in loc de un singur arbore antreneaza mai multi arbori pe subseturi aleatorii de date si features, agregand predictiile lor. Aceasta abordare reduce varianța fata de un DT izolat. Am testat 50, 100 si 200 de arbori, pastrand cel mai bun max_depth gasit anterior.

Random Forest agraga predictiile mai multor arbori pentru a reduce varianta si a imbunatati generalizarea.

Regresie:

Baseline — Regresie Liniara:

Modelul baseline este o regresie liniara simpla, fara regularizare. Acesta obtine urmatoarele performante:

- MAE (validare): 6289
- MSE (validare): 66,341,004
- RMSE (validare): 8145
- R^2 (validare): 0.9522
- R^2 (train): 0.9524

Experiment 1 — Ridge (regularizare L2):

	R ² train (↑)	R ² val (↑)	MAE val (↓)
alpha			
0.010000	0.952400	0.952200	6289.000000
0.100000	0.952400	0.952200	6289.000000
1.000000	0.952400	0.952200	6289.000000
10.000000	0.952400	0.952200	6289.000000
100.000000	0.952200	0.952100	6295.000000
1000.000000	0.937800	0.937900	7125.000000

Ridge adauga o penalizare proportionala cu patratul coeficientilor, reducandu-i pe toti proportional fara sa ii anuleze. Am testat alpha in [0.01, 0.1, 1.0, 10.0, 100.0, 1000.0]. Rezultatele din table arata ca:

- alpha intre 0.01-10.0: performanta identica ($R^2=0.9522$, MAE=6289) - regularizarea este insuficienta pentru a avea impact
- alpha=100.0: incepe o usoara degradare ($R^2=0.9521$, MAE=6295)
- alpha=1000.0: degradare clara ($R^2=0.9378$, MAE=7125) - underfit din cauza penalizarii prea puternice

Experiment 2 — Lasso (regularizare L1):

	R ² train (↑)	R ² val (↑)	MAE val (↓)	Coef != 0
alpha				
0.010000	0.952400	0.952200	6289.000000	41
0.100000	0.952400	0.952200	6289.000000	41
1.000000	0.952400	0.952200	6289.000000	41
10.000000	0.952400	0.952200	6289.000000	36
100.000000	0.950700	0.950600	6398.000000	29
1000.000000	0.846100	0.846100	11594.000000	16

Lasso penalizeaza suma valorilor absolute ale coeficientilor, ceea ce duce la anularea completa a unor coeficienti — practic o selectie automata de attribute. Am testat aceleasi valori de alpha. Cel mai bun alpha pentru Lasso a fost **10.0**.

Rezultatele din table arata ca:

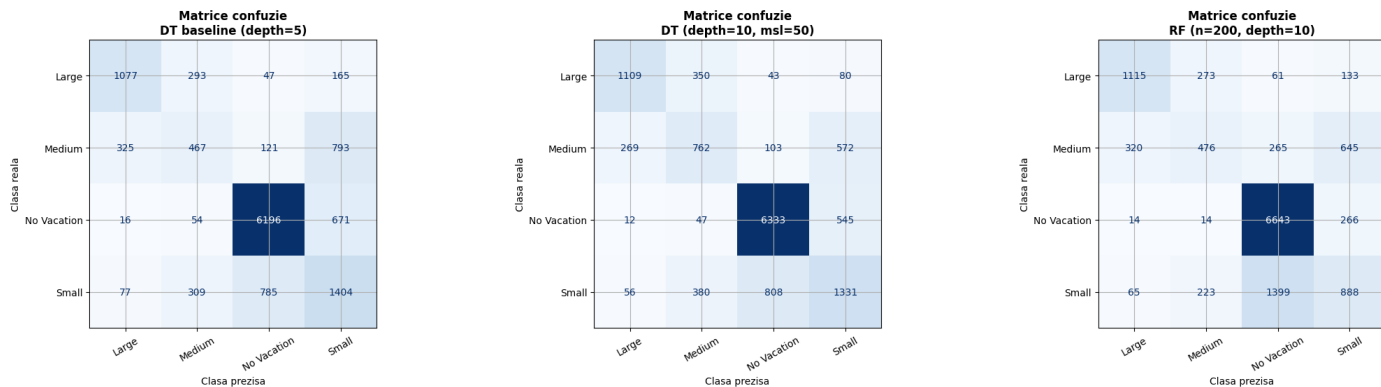
- alpha intre 0.01-1.0: performanta identica cu baseline ($R^2=0.9522$, MAE=6289), toti cei 40 de coeficienti sunt nenulli
- alpha=10.0: performanta identica dar 35 coeficienti ramasi (5 eliminati automat)
- alpha=100.0: scadere performanta ($R^2=0.9507$, MAE=6398), doar 28 coeficienti ramasi
- alpha=1000.0: underfitting sever ($R^2=0.8461$, MAE=11594), doar 15 coeficienti ramasi

Evaluarea algoritmilor:

Comparatie algoritmi de clasificare:

	P_Large (!)	R_Large (!)	F1_Large (!)	P_Medium (!)	R_Medium (!)	F1_Medium (!)	P_No Vacation (!)	R_No Vacation (!)	F1_No Vacation (!)	P_Small (!)	R_Small (!)	F1_Small (!)
Model												
DT baseline (depth=5)	0.7204	0.6808	0.7000	0.4159	0.2737	0.3302	0.8667	0.8932	0.8797	0.4629	0.5452	0.5007
DT (depth=10, msl=50)	0.7669	0.7010	0.7325	0.4951	0.4467	0.4696	0.8691	0.9129	0.8905	0.5265	0.5169	0.5217
RF (n=200, depth=10)	0.7365	0.7048	0.7203	0.4828	0.2790	0.3536	0.7939	0.9576	0.8681	0.4596	0.3449	0.3941

Fig 8. Matrici de confuzie - toate modelele de clasificare



Am comparat performanta celor trei modele pentru fiecare clasa de vacanta. DT optimizat (depth=10, msl=50) obtine cele mai bune rezultate pe 3 din 4 clase, cu imbunatatiri semnificative fata de baseline (de exemplu, pentru Medium: F1 creste de la 0.33 la 0.47). Clasa "No Vacation" este cea mai bine prezisa de toate modelele (F1 peste 0.86), in timp ce clasa "Medium" ramane cea mai dificila (F1 maxim 0.47).

Matricile de confuzie confirma ca DT optimizat are diagonala cea mai pronuntata si mai putine confuzii intre clase. Random Forest obtine Recall foarte mare pe "No Vacation" (0.9576) dar Precision mai mica, sugerand ca supraevalueaza aceasta clasa.

Alegerea DT optimizat (depth=10, msl=50) ca model final pentru submitie a fost justificata de performanta superioara fata de Random Forest: F1=0.7439 (DT) vs F1=0.6600 (RF). Aceasta diferenta se explica prin faptul ca max_depth=10 a fost ales specific pentru DT. In cazul RF, limita de adancime restrictioneaza fiecare arbore individual, impiedicand ansamblul sa exploateze toata puterea de expresie. Astfel, DT optimizat ofera cel mai bun raport performanta/complexitate pentru acest set de date.

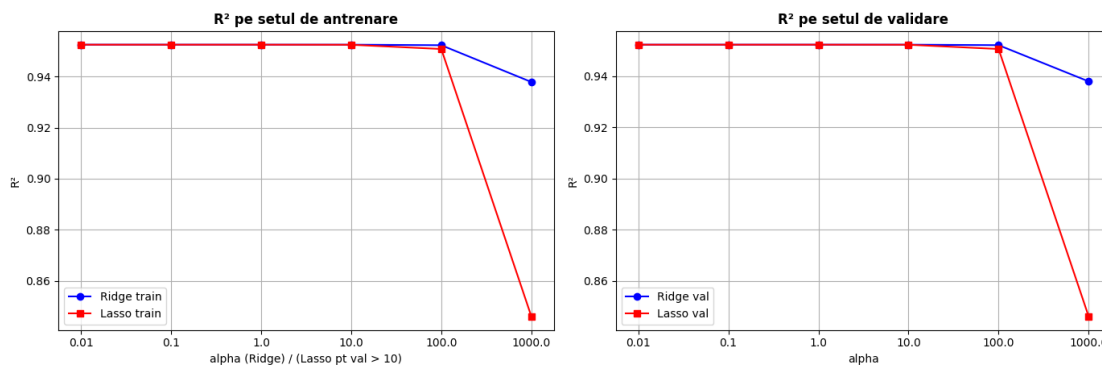
Comparatie algoritmi de regresie:

Tabel comparative algoritmi de regresie:

	MAE val (!)	MSE val (!)	RMSE val (!)	R ² val (!)	R ² train (!)
Model					
Linear Regression	6289.000000	66341004.000000	8145.000000	0.952200	0.952400
Ridge (alpha=1.0)	6289.000000	66340658.000000	8145.000000	0.952200	0.952400
Lasso (alpha=1.0)	6289.000000	66339953.000000	8145.000000	0.952200	0.952400

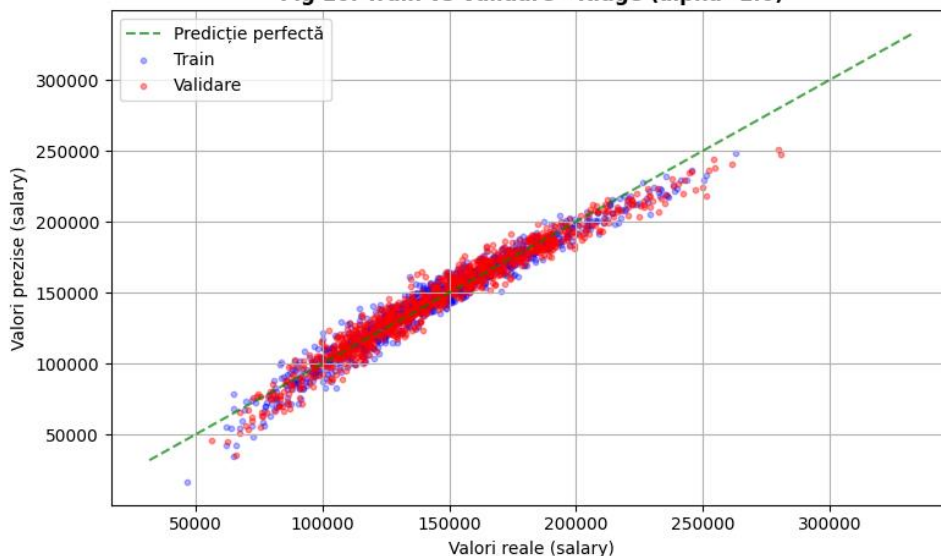
Privind rezultatele din tabelul comparativ, toate cele trei modele — Regresia Liniară, Ridge și Lasso — au obținut rezultate **aproape identice**, cu $R^2=0.9522$ pe setul de validare și $MAE \sim 6289$ USD. Acest lucru sugerează că datele au o structură liniară foarte clară, iar regularizarea nu aduce un beneficiu semnificativ în acest caz.

Fig 9. Evoluție R^2 Ridge vs Lasso în funcție de alpha



Din **Fig 9** se observă că R^2 rămâne stabil pentru valori mici și medii de alpha, atât pentru Ridge cât și pentru Lasso. Diferența apare la valori mari de alpha — Lasso devine instabil mult mai repede decât Ridge, cu o scădere mare la $\alpha=1000$, în timp ce Ridge își păstrează performanța până la $\alpha=100$.

Fig 10. Train vs Validare - Ridge ($\alpha=1.0$)



Din **Fig 10** se observă că punctele de train și validare sunt distribuite foarte similar în jurul dreptei de predicție perfectă, ceea ce confirmă că modelul nu face overfit. Erorile mai mari apar la capetele distribuției — pentru salarii foarte mici sau foarte mari — ceea ce era de așteptat, deoarece acestea sunt și cele mai rare în date.